Home Assignment: Single-Agent ADK Data Query Agent

Objective

Build a single-agent system using Google ADK that answers filtering questions about a sensor dataset. The agent should interpret natural language queries, translate them into structured tool calls, and return filtered results.

Important: The agent should **only respond to filtering questions**. It should not perform calculations, aggregations, or general conversation.

Dataset

You will be provided with `sensor_data.xlsx` containing multiple rows with the following columns:

- `asset_id` (string): Equipment identifier
- `location` (string): Geographic area (e.g., "Area A", "Zone 2")
- `timestamp` (datetime): When the reading was taken
- `sensor_type` (string): Type of sensor ("temperature", "pressure", "gas", "vibration")
- `value` (float): Numeric sensor reading
- `anomaly_label` (boolean): Whether this reading is flagged as anomalous

---

Required Tasks

1. Set Up Google ADK

- Create a Python project using Google ADK
- Use pixi for dependency management
- Define a single agent (no multi-agent setup required)

---

2. Implement a Dataset Filtering Tool

Create **one tool** that applies **a single filter** to either the full dataset or to previously filtered results.

Important Design Constraint: The tool accepts only one filter criterion per call. Therefore, for a query such as “Show gas anomalies in Area A with value above 50,” the agent should call the tool multiple times: first filter by location, then sensor type, then anomaly status, then value range.

Tool Signature:

python
```
filter_records(
    filter_type: str,
    filter_value: Union[str, float, dict],
    previous_results: Optional[List[dict]] = None
) -> List[dict]
```

Parameters:

- filter_type: One of "location", "sensor_type", "value_range", or "anomaly_label"
- filter_value:
    - For "location" or "sensor_type": a string (e.g., "Area A", "gas")
    - For "value_range": a dict with optional "min" and/or "max" keys (e.g., {"min": 50}, {"min": 20, "max": 30})
    - For "anomaly_label": boolean True
- previous_results: Optional list of records from a previous filter call
    - If None: filter operates on the full dataset
    - If provided: filter operates only on these records
- Expected Behavior:
    - If previous_results is None, load sensor_data.xlsx.
    - Otherwise, use previous_results as the input dataset.
    - Apply exactly one filter based on filter_type.
    - Return the filtered records as a list of dictionaries.
    - Do not apply a row limit during intermediate filtering.
    - The final answer may display only the first 20 records.
    - If no results match, return an empty list.

3. Build the Agent

Your agent must:

1. Accept a natural language query from the user
2. Determine if the query is a **filtering question** or not
3. If it's a filtering question:
    - Convert the query into structured arguments for `filter_records`
    - Call the tool
    - Return a clear, human-readable answer based on the filtered results
4. If it's **not** a filtering question:
    - Politely decline and explain what types of questions it can answer

Example Valid Queries (should work):

- "Show gas readings in Area A"
- "Show gas anomalies in Zone 2 with value above 60"
- "Find vibration anomalies in Area A with value above 6"
- "Show temperature sensors with values between 20 and 30"
- "Show gas readings above 50 that are not anomalies"

Example Invalid Queries (should decline):

- "How many anomalies are in Area B?" (requires counting)
- "What is the average temperature?" (requires calculation)

4. Control Agent Execution

Run your agent with **different execution configurations** and observe behavior.

In ADK, you can control agent execution using parameters like:

- `max_iterations` (for LoopAgent)
- Model temperature
- Or other configuration parameters available in ADK

Experiment with at least 2 different configurations, for example:

- Configuration A: `max_iterations=1` (single-shot reasoning)
- Configuration B: `max_iterations=5` (iterative refinement allowed)

5. Write a Short Report

Submit a brief report (1-2 pages) covering:

1. **Agent Architecture**
2. **Tool Design**
3. **Example Queries and Outputs**
4. **Failure Cases**
5. **Execution Control Observations**
    - What happened when you changed `max_iterations` or other parameters?
    - Did it affect accuracy? Efficiency? Behavior?
6. **What You Would Improve Next**
    - Given more time, what would you enhance?

Optional but Encouraged - Assignment (Design Only): RL Framework for Agent Optimization

We are interested in how you would apply reinforcement learning to optimize agent parameters automatically.

Goal

Design an RL framework where the agent learns the optimal value of `max_iterations` (or similar execution control parameter) to balance correctness and efficiency.

What to Describe

1. State

2. Action

3. Reward Function

4. Policy

5. Training Setup

6. Evaluation

Submission Requirements

How to Submit

- **GitHub repository** (public or grant us access)
- **README.md** must include:
    - Setup instructions (how to install and run)
    - How to test the agent
    - Any assumptions or design decisions

Optional: RL Design Document

If you complete the optional section, include:

- `rl_design.pdf` or `rl_design.md` with your RL framework design